iamneo



Data Aggregation, Joins, and Unions

Introduction to Data Aggregation

Data aggregation is the process of gathering **data** from multiple sources and combining it into a **unified view**.

- Used for **data analysis** and **reporting**
- Involves **grouping** and **aggregating** data

DataFrames are commonly used for data aggregation in **Spark**.

Aggregation helps in **summarizing large datasets** into **meaningful insights**.

Common aggregation functions include **count**, **sum**, **avg**, **min**, and **max**.

Data aggregation is crucial for **business intelligence** and **data-driven decision-making**.

Performing Aggregations on DataFrames

DataFrames provide various **aggregation methods** like **groupBy** and **agg**.

groupBy is used to **group data** based on one or more **columns**.

agg is used to apply **aggregation functions** on **grouped data**.

You can perform **multiple aggregations** in a single operation.

Aggregations can be performed on **numeric columns** using functions like **sum** and **avg**.

Example: Performing Aggregations on DataFrames

Example of Aggregation on DataFrame

```python
from pyspark.sql import SparkSession

# Initialize SparkSession
spark = SparkSession.builder.appName("DataFrameAgg").getOrCreate()

# Create DataFrame from Orders data
data = [(1, "Keyboard", 400, 4), (2, "Mouse", 300, 4), (3, "Monitor", 12000, 3)]
df = spark.createDataFrame(data, ["order_id", "item", "amount", "customer_id"])

# Group by customer_id and aggregate
result = df.groupBy("customer_id").agg({"amount": "sum", "order_id": "count"})

# Output result
print("output:")
result.show()
```

```
output:
+-----------+-----------+---------------+
|customer_id|sum(amount)|count(order_id)|
+-----------+-----------+---------------+
|          4|        700|              2|
|          3|      12000|              1|
+-----------+-----------+---------------+
```

Computing Statistics with DataFrames

DataFrames provide methods to **compute statistics** like **mean**, **stddev**, and **variance**.

describe method is used to **generate summary statistics**.

You can compute **statistics** for **numeric columns**.

corr method is used to compute the **correlation** between two columns.

Statistics are crucial for **understanding data distribution** and **identifying trends**.

Example: Computing Statistics with DataFrames

Computing Statistics Example

```python
from pyspark.sql import SparkSession

# Initialize SparkSession
spark = SparkSession.builder.appName("DataFrameStats").getOrCreate()

# Create DataFrame from Orders data
data = [(1, "Keyboard", 400, 4), (2, "Mouse", 300, 4), (3, "Monitor", 12000, 3
df = spark.createDataFrame(data, ["order_id", "item", "amount", "customer_id"]

# Compute statistics
stats = df.describe("amount")
corr = df.corr("amount", "customer_id")

# Output result
print("output:")
stats.show()
print("Correlation:", corr)
```

```
output:
+-------+------------------+
|summary|            amount|
+-------+------------------+
|  count|                 3|
|   mean|4233.333333333333|
| stddev|6726.316475853135|
|    min|               300|
|    max|             12000|
+-------+------------------+

Correlation: -0.9999723712384538
```

Using groupBy for Data Grouping

groupBy is used to **group data** based on one or more **columns**.

After grouping, you can apply **aggregation functions**.

groupBy returns a **GroupedData object**.

You can **aggregate** grouped data using functions like **sum** and **count**.

groupBy is a powerful operation for **data analysis** and **reporting**.

Example: Using groupBy for Data Grouping

Example of groupBy

```
from pyspark.sql import SparkSession

# Initialize SparkSession
spark = SparkSession.builder.appName("DataFrameGroupBy").getOrCreate()

# Create DataFrame from Orders data
data = [(1, "Keyboard", 400, 4), (2, "Mouse", 300, 4), (3, "Monitor", 12000, 3)]
df = spark.createDataFrame(data, ["order_id", "item", "amount", "customer_id"])

# Group by customer_id and aggregate
result = df.groupBy("customer_id").agg({"amount": "sum", "item": "count"})

# Output result
print("output:")
result.show()
```

```
output:
+-----------+-----------+-----------+
|customer_id|sum(amount)|count(item)|
+-----------+-----------+-----------+
|          4|        700|          2|
|          3|      12000|          1|
+-----------+-----------+-----------+
```

Aggregation Functions: count and sum

count is used to **count the number of rows** in a DataFrame or a group.

sum is used to **compute the sum** of a numeric column.

count and **sum** are commonly used for **data aggregation**.

These functions can be used with **groupBy** for **grouped aggregations**.

They are essential for **data analysis** and **reporting**.

Example: Aggregation Functions: count and sum

Example of count and sum

```python
from pyspark.sql import SparkSession

# Initialize SparkSession
spark = SparkSession.builder.appName("DataFrameCountSum").getOrCreate()

# Create DataFrame from Orders data
data = [(1, "Keyboard", 400, 4), (2, "Mouse", 300, 4), (3, "Monitor", 12000, 3)]
df = spark.createDataFrame(data, ["order_id", "item", "amount", "customer_id"])

# Group by customer_id and apply count, sum
result = df.groupBy("customer_id").agg({"amount": "sum", "order_id": "count"})

# Output result
print("output:")
result.show()
```

```
output:
+-----------+-----------+---------------+
|customer_id|sum(amount)|count(order_id)|
+-----------+-----------+---------------+
|          4|        700|              2|
|          3|      12000|              1|
+-----------+-----------+---------------+
```

Aggregation Functions: avg, min, max

avg is used to **compute the average** of a numeric column.

min and **max** are used to **find the minimum and maximum values**.

These functions are used for **data analysis** and **summarization**.

They can be used with **groupBy** for **grouped aggregations**.

These aggregations provide **insights into data trends** and **patterns**.

Example: Aggregation Functions: avg, min, max

Example of avg, min, max

```python
from pyspark.sql import SparkSession, functions as F

# Initialize SparkSession
spark = SparkSession.builder.appName("DataFrameAgg").getOrCreate()

# Create DataFrame from Orders data
data = [(1, "Keyboard", 400, 4), (2, "Mouse", 300, 4), (3, "Monitor", 12000, 3)]
df = spark.createDataFrame(data, ["order_id", "item", "amount", "customer_id"])

# Group by customer_id and apply avg, min, max
result = df.groupBy("customer_id").agg(F.avg("amount"), F.min("amount"), F.max("amount"))

# Output result
print("output:")
result.show()
```

```
output:
+-----------+-----------+-----------+-----------+
|customer_id|avg(amount)|min(amount)|max(amount)|
+-----------+-----------+-----------+-----------+
|          4|      350.0|        300|        400|
|          3|    12000.0|      12000|      12000|
+-----------+-----------+-----------+-----------+
```

Understanding Joins in Spark

Joins are used to **combine data** from two DataFrames based on a **common column**.

Spark supports various types of **joins**, including **inner**, **left**, **right**, and **outer**.

join is used to perform a **join operation** between two DataFrames.

The **join type** determines how the DataFrames are combined.

Joins are essential for **combining data** from multiple sources.

Inner Join Operations

Inner join returns only the **rows that have a match** in both DataFrames.

It is used to **combine data** based on a **common column**.

inner join is the default join type in Spark.

You can perform an **inner join** using the **join** method.

Inner joins are useful for **retrieving related data**.

Example: Inner Join Operations

Example of Inner Join

```
from pyspark.sql import SparkSession

# Initialize SparkSession
spark = SparkSession.builder.appName("InnerJoin").getOrCreate()

# Create DataFrames from Customers and Orders data
cust_data = [(1, "John", 31), (2, "Robert", 22), (4, "John", 25)]
order_data = [(1, "Keyboard", 400, 4), (2, "Mouse", 300, 4), (3, "Monitor", 12000, 3)]
cust_df = spark.createDataFrame(cust_data, ["customer_id", "name", "age"])
order_df = spark.createDataFrame(order_data, ["order_id", "item", "amount", "customer_id"])

# Perform inner join
result = cust_df.join(order_df, "customer_id", "inner")

# Output result
print("output:")
result.select("name", "item", "amount").show()
```

```
output:
+----+--------+------+
|name|    item|amount|
+----+--------+------+
|John|Keyboard|   400|
|John|   Mouse|   300|
+----+--------+------+
```

Left and Right Joins

Left join returns all the **rows from the left DataFrame** and the **matched rows from the right DataFrame**.

Right join is similar to **left join** but returns all the **rows from the right DataFrame**.

These joins are used to **combine data** from two DataFrames.

You can perform **left** and **right joins** using the **join** method.

They are useful for **retrieving related data**.

Example: Left and Right Joins

Example of Left and Right Joins

```python
from pyspark.sql import SparkSession

# Initialize SparkSession
spark = SparkSession.builder.appName("LeftRightJoin").getOrCreate()

# Create DataFrames from Customers and Orders data
cust_data = [(1, "John", 31), (2, "Robert", 22), (5, "Betty", 28)]
order_data = [(1, "Keyboard", 400, 4), (2, "Mouse", 300, 4), (3, "Monitor", 12000, 3)]
cust_df = spark.createDataFrame(cust_data, ["customer_id", "name", "age"])
order_df = spark.createDataFrame(order_data, ["order_id", "item", "amount", "customer_id"])

# Perform left and right joins
left_result = cust_df.join(order_df, "customer_id", "left")
right_result = cust_df.join(order_df, "customer_id", "right")

# Output results
print("output:")
print("Left Join:")
left_result.select("name", "item", "amount").show()
print("Right Join:")
right_result.select("name", "item", "amount").show()
```

```
output:
Left Join:
+------+----+------+
|  name|item|amount|
+------+----+------+
|  John|NULL|  NULL|
| Betty|NULL|  NULL|
|Robert|NULL|  NULL|
+------+----+------+

Right Join:
+----+--------+------+
|name|    item|amount|
+----+--------+------+
|NULL|Keyboard|   400|
|NULL| Monitor| 12000|
|NULL|   Mouse|   300|
+----+--------+------+
```

Outer Join Applications

Outer join returns all the **rows from both DataFrames**.

It is used to **combine data** from two DataFrames.

outer join includes all records from both DataFrames.

You can perform an **outer join** using the **join** method.

Outer joins are useful for **identifying missing data**.

Example: Outer Join Applications

Example of Outer Join

```python
from pyspark.sql import SparkSession

# Initialize SparkSession
spark = SparkSession.builder.appName("OuterJoin").getOrCreate()

# Create DataFrames from Customers and Orders data
cust_data = [(1, "John", 31), (2, "Robert", 22), (5, "Betty", 28)]
order_data = [(1, "Keyboard", 400, 4), (2, "Mouse", 300, 4), (3, "Monitor", 12000, 3)]
cust_df = spark.createDataFrame(cust_data, ["customer_id", "name", "age"])
order_df = spark.createDataFrame(order_data, ["order_id", "item", "amount", "customer_id"])

# Perform outer join
result = cust_df.join(order_df, "customer_id", "outer")

# Output result
print("output:")
result.select("name", "item", "amount").show()
```

```
output:
+------+--------+------+
|  name|    item|amount|
+------+--------+------+
|  John|    NULL|  NULL|
|Robert|    NULL|  NULL|
|  NULL| Monitor| 12000|
|  NULL|Keyboard|   400|
|  NULL|   Mouse|   300|
| Betty|    NULL|  NULL|
+------+--------+------+
```

Combining Datasets with Union

Union is used to **combine DataFrames** by **stacking them on top of each other**.

It requires the DataFrames to have the **same schema**.

union is different from **join** as it **combines data vertically**.

You can use **union** to **concatenate DataFrames**.

union is useful for **combining datasets** with the same structure.

Example: Combining Datasets with Union

Example of Union

```python
from pyspark.sql import SparkSession

# Initialize SparkSession
spark = SparkSession.builder.appName("DataFrameUnion").getOrCreate()

# Create DataFrames from Customers data
data1 = [(1, "John", 31), (2, "Robert", 22)]
data2 = [(3, "David", 22), (4, "John", 25)]
df1 = spark.createDataFrame(data1, ["customer_id", "name", "age"])
df2 = spark.createDataFrame(data2, ["customer_id", "name", "age"])

# Perform union
result = df1.union(df2)

# Output result
print("output:")
result.show()
```

```
output:
+-----------+------+---+
|customer_id|  name|age|
+-----------+------+---+
|          1|  John| 31|
|          2|Robert| 22|
|          3| David| 22|
|          4|  John| 25|
+-----------+------+---+
```

Union vs. Join Operations

Union and **join** are both used to **combine data**, but in different ways.

Union combines DataFrames **vertically**.

Join combines DataFrames **horizontally** based on a common column.

The choice between **union** and **join** depends on the **data structure** and the **analysis goal**.

Understanding the difference is crucial for **effective data manipulation**.

Joins	UNION
JOIN combines data from many tables based on a matched condition between them.	SQL combines the result-set of two or more SELECT statements.
It combines data into new columns.	It combines data into new rows
Number of columns selected from each table may not be same.	Number of columns selected from each table should be same.
Datatypes of corresponding columns selected from each table can be different.	Datatypes of corresponding columns selected from each table should be same.
It may not return distinct columns.	It returns distinct rows.

Summary of Aggregation and Joins

Data **aggregation** involves **grouping** and **aggregating** data using functions like **sum** and **avg**.

Joins are used to **combine data** from multiple DataFrames based on a common column.

Understanding **aggregation** and **joins** is crucial for **data analysis** and **reporting**.

These techniques are essential for **data-driven decision-making**.

Mastering **aggregation** and **joins** will enhance your **data manipulation skills**.